Movie Prediction App: 3-Week Project Plan

Oct 19, 2026 · @Dimitris

Overview

This plan builds a full-stack app in 15 working days (about 3 weeks): creators enter a movie's parameters and get ranges for cost, revenue and score, while the public browses movie information.

- **Problem:** creators lack a quick, data-backed way to sanity-check a project before pitching it, and audiences lack one easy place to explore movies.
- **Users:** movie creators (prediction form) and the general public (browse and search).
- **Outputs:** estimated budget range, revenue range and score range, each with the top factors behind it.
- **Stack:** Python, scikit-learn/LightGBM, FastAPI, SQLite, React (Vite), Docker. Data from the TMDB API and the Kaggle TMDB 5000 dataset.
- **Each week ends with a demoable result**, so a working project exists even if time runs short.

Week 1: Data and models

Week 1 ends with a script that trains the models and reports honest metrics against baselines.

Day	Task	Details
1	Setup and data collection	Create the repo with `/backend`, `/ml`, `/frontend` and a README skeleton. Get a TMDB API key and write a script pulling details, credits, genres, release dates, budget, revenue and ratings. Download the Kaggle TMDB dataset as a fallback for older films.
2	Cleaning	Drop rows with zero or missing budget or revenue, and remove duplicates. Adjust for inflation or add release year as a feature. Store clean data in SQLite.

Day	Task	Details
3	Feature engineering	Genres, runtime, release month and year, language, production company, director, top 3 billed actors. Build director and actor track-record features from films released before the target film only. Exclude `popularity`, `vote_count` and `vote_average`, which exist only after release.
4	Baselines and evaluation	Split by time: train up to 2015, validate 2016-2018, test 2019 onward. Build simple baselines such as median per genre.
5	First models	Train LightGBM models for log-revenue and score, with quantile versions for ranges. Record metrics in a table for the README.

Deliverable: a notebook or script that trains the models and reports metrics against baselines.

Cut line: if behind, skip the budget model and keep only revenue and score.

Week 2: Backend and the creator tool

Week 2 ends with a creator able to submit the form and see raw prediction results from the API.

Day	Task	Details
6	Budget estimate model	Train a model that estimates a budget range from genre, cast, director and release info. If it performs poorly, show a wide range labelled "rough estimate".
7	Explanations and packaging	Add SHAP or feature importance for each prediction. Save models with joblib and write one `predict()` function returning budget, revenue and score ranges plus top drivers.
8	FastAPI backend	Endpoints: `POST /predict`, `GET /movies` (search, filter, paginate), `GET /movies/{id}`, `GET /meta/genres`. Validate inputs with Pydantic.

Day	Task	Details
9	Tests and Docker	Test the prediction endpoint and feature pipeline, especially the no-leakage rule. Add a Dockerfile and docker-compose for backend and database.
10	Frontend and creator form	Set up React (Vite) with a router. Build the form: director and actor autocomplete, genre multi-select, runtime, release month, language, optional budget.

Deliverable: a creator can submit the form and get prediction results from the API.

Cut line: the public pages have not started yet, and that is fine.

Week 3: UI polish, public side and delivery

Week 3 ends with a deployed app, a clean repo and a demo story.

Day	Task	Details
11	Results page	Show budget, revenue and score as ranges with a simple range chart. Show the top 5 factors pushing the prediction up or down in plain English. Add a clear note on limitations.
12	Public browse page	Movie grid with search and filters (genre, year). Movie detail page with poster, cast, crew and facts. Include the required TMDB attribution notice and logo.
13	Compare and edge cases	Add a "try changing one thing" option, such as a different genre or release month. Handle unknown actors and directors gracefully, with loading and error states.
14	Deployment	Deploy backend and frontend (for example Render, Fly.io or Railway free tiers). Confirm the live link works end to end.

Day	Task	Details
15	Documentation and demo prep	Write the README: problem statement, architecture diagram, how to run, evaluation results, limitations, next steps. Prepare a 5-minute demo script and a 1-page summary for the interviewer.

Deliverable: a deployed app, a clean repo and a demo story.

Rules to protect the timeline

Four rules keep the project inside 3 weeks.

1. Time-box data cleaning to 2 days; it is the most common place projects stall.
2. If behind, cut in this order: the compare feature, then public filters, then the budget model, then SHAP charts (keep a text version).
3. Do not add accounts, reviews, trailer analysis or chatbots.
4. Commit daily so the repo history shows steady progress.

Definition of done

The project is done when all four checks below pass.

- [ ] A creator can fill in the form and get ranges for cost, revenue and score with explanations.
- [ ] A visitor can search and view movies.
- [ ] The model is evaluated on a time-based split against a baseline, and the README reports the numbers honestly.
- [ ] The app is live, with TMDB attribution and limitations clearly shown.